SYNERGETICS
GET IT RIGHT

Java 11 New Features

Makarand Bhoir

Microsoft Partner | Silver Learning Gold Cloud Platform

Microsoft Cloud Solution Provider

Educate | Advise | Implement | Manage

Agenda

- Local Variable Type Inference (LVTI)
- Private Method in Interface
- Modules
- Annotations
- Secure Coding

Local Variable Type Inference

SYNERGETICS
GET IT RIGHT

- Java is a strongly typed language i.e., you must specify the type of a variable when introducing it.
- However, since Java 10, we have the option, in certain scenarios, of using the keyword var instead of the type. In these situations, the compiler infers the type e.g., var x = “ abc ”; // x is a String
- Remember that the variables must be local and must be initialized on the line where they are declared.

Local Variable Type Inference

- LVTI.java Example

Private method in interface

- Introduced in Java 9, interfaces can now have private methods.
- These private methods can be both static and non static.
- As they are private , they are accessible within the interface only.
- As with classes, you cannot access a non static interface method from a static method.

Private method in interface

- Benefits are:
  - reduce code duplication
  - improve code encapsulation

Private method in interface

- PrivateMethodInterface.java Example

Modules

SYNERGETICS
GET IT RIGHT

- Modules, introduced in Java 9, provide an extra layer of encapsulation by enabling us to group related packages.
- From a high level, modules enable us to specify well defined boundaries/dependencies in our code base.
- As a developer, you can specify which packages are accessible outside the module and also the module dependencies

Why Modules?

SYNERGETICS
GET IT RIGHT

- Improved access control:
  - In addition to private, package, protected and public now we have the ability to restrict packages to certain other packages.

- Improved large scale structure of applications:
  - boundaries/dependencies in your code base.
  - hide implementation details.
  - improved decoupling.

Modules

- By default, all classes in the packages in a module are strongly encapsulated i.e. no other module can use these classes, even if they are public classes.

- Package names and module names live in separate namespaces i.e. you can have the same identifiers without any conflict (name clash).

- A module contains one or more packages plus a module info.java file.

Services

- Modules are tightly coupled due to the use of requires , exports and the knowledge and creation of specific types.
- Services enable the decoupling of modules.
- Rather than modules directly coupling to other modules, there is now a middle layer, the service registry which abstracts the consumers from the producers.
- The service (interface) contract , which is hosted by the registry, is what binds the consumers and producers together.

Annotations

- Metadata is information about information.
- Annotations, via metadata, enable us to add value to our code.
- We can annotate (assign metadata) to classes, methods variables etc..
- Though optional, when used they must be used correctly.

Annotations

- Annotations operate much like interfaces. In fact, their origins are in interfaces. Their “elements” look like abstract methods.
- Annotation names are case sensitive; however, it is common practice to start the name with an uppercase letter.
- Like interfaces, annotations can be applied to unrelated classes.

Annotations

SYNERGETICS
GET IT RIGHT

- As stated earlier, annotations have a lot in common with interfaces; for example, a marker annotation has no elements (a marker interface has no methods). Annotations, as with interfaces, can be applied to unrelated classes.

- In fact, we annotate our annotation with @interface

- public @interface MyAnnotation {} // marker annotation

Annotations

- An annotation element is an attribute of the annotation. The elements (attributes) can have values.

```
public @interface MyAnnotation{
        int myElement (); // looks like an abstract
}
```

- Remember that annotations have their origins in interfaces. Essentially, the JVM translates the above element into an interface method and the annotation itself as an implementation of the interface.

Annotations

- There are annotations that can be applied to annotations
- As opposed to applying built in annotations to types (methods, interfaces, fields etc..); we can actually apply built in annotations to annotations themselves.
- For example, you might want to limit where your annotation can be used or you may want your annotation information included in Javadoc documentation.

Annotations

- @Target limits the types that the annotation can be applied to.
- The types are specified as an array of ElementType enum values.

Annotations

SYNERGETICS
GET IT RIGHT

ElementType value	Scope (what it applies to)
TYPE	Interfaces, enums, classes, annotations.
METHOD	Method declarations
PARAMETER	Constructor and method parameters
FIELD	Instance and static variables
CONSTRUCTOR	Constructor declarations
LOCAL_VARIABLE	Local variables
ANNOTATION_TYPE	Annotations
TYPE_USE	Anywhere there is a Java data type. This includes where types are *used* e.g. object creation with *new*.

Annotations

- In generics, we encountered “type safety” where certain information is discarded by the compiler when converting source code into .class file.
- Similarly, annotations may be discarded at compile time, at runtime or not at all. We can control when this happens by using the @Retention annotation.
- We specify the level of retention using an enum value from RetentionPolicy

Annotations

RetentionPolicy value	Description
SOURCE	Source file only, compiler discards it.
CLASS	Stored in the .class file but not available at runtime. This is the default compiler behaviour.
RUNTIME	Stored in the .class and available at runtime (via reflection).

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.SOURCE)  // annotation discarded by the compiler
@interface Mouse{}                  // i.e. not in .class file

@Retention(RetentionPolicy.RUNTIME) // annotation stored in .class file and
@interface Keyboard{}               // available at runtime (via reflection)
```

Data Integrity

SYNERGETICS
GET IT RIGHT

Accessibility

- limit access as much as possible --“principle of least privilege”.
- instance variables/methods should be private

Restrict extensibility

- prevent subclassing by marking the class as final

Data Integrity

- Immutable objects are objects that cannot be changed after Creation.
- They are secure objects and use the following guidelines:
  - Do not provide any “setter” methods.
  - Make all the fields private and final
  - Prevent subclassing (prevents overriding):
    - make the class final
    - make the constructor private and provide a public static factory method e.g. “createNewInstance”
  - Prevent subclassing (prevents overriding):
    - immutable types e.g. String , ok
    - mutable types e.g. StringBuilder , do NOT share references
    - i.e . use “defensive copying” and “advanced encapsulation”

Common Injection

- SQL Injection
  - where user input retrieves unexpected results.
  - protection provided by PreparedStatement with bind variables (and not Statement).

SYNERGETICS

GET IT RIGHT

Thank You

Educate | Advise | Implement | Manage